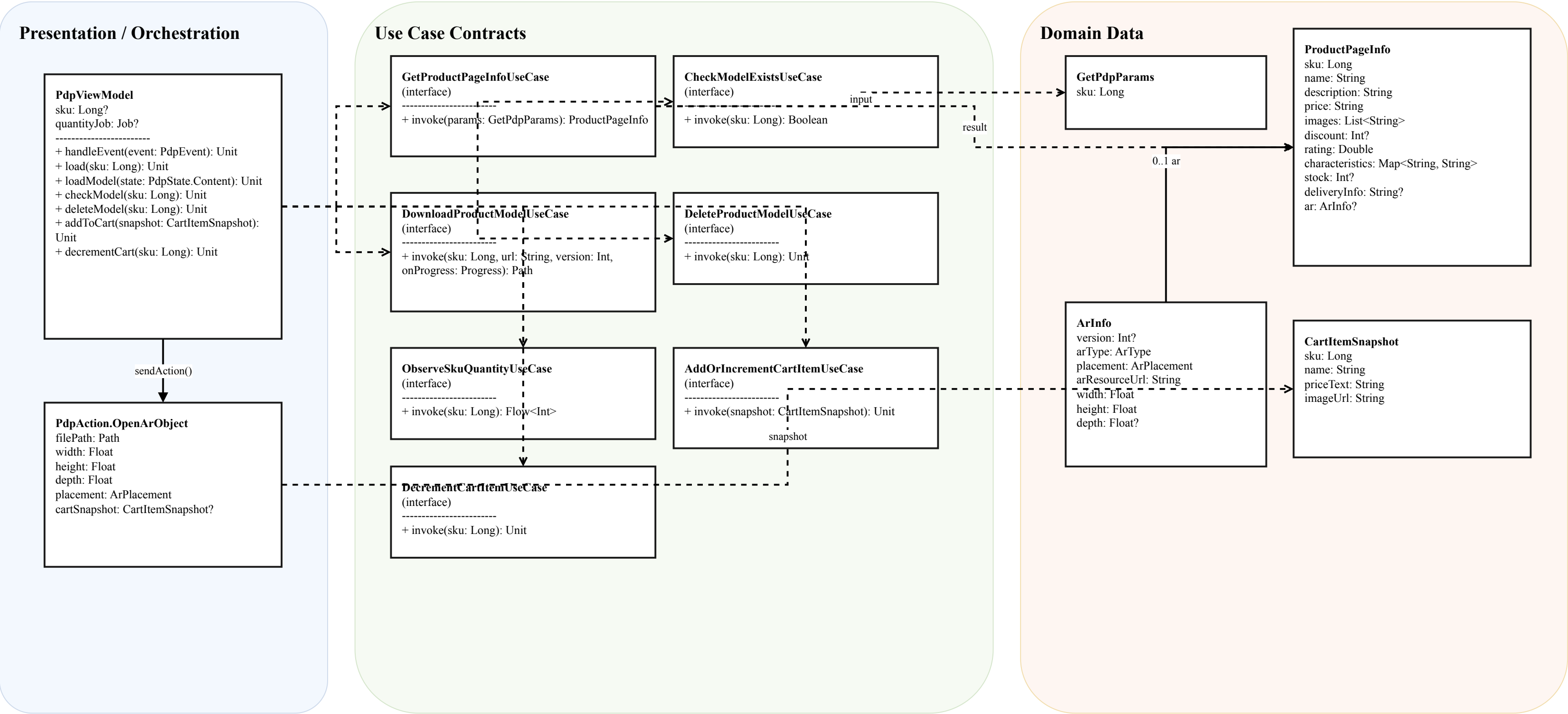


UML Core Domain - PDP AR Flow

Блок 3. Ключевой сценарий: карточка товара - кэш 3D-модели - запуск AR - синхронизация корзины

Смысл сценария
Пользователь открывает PDP, приложение загружает ProductPageInfo, проверяет локальный кэш 3D-модели, при необходимости скачивает файл, затем формирует одноразовое действие OpenArObject и одновременно поддерживает количество товара в локальной корзине.

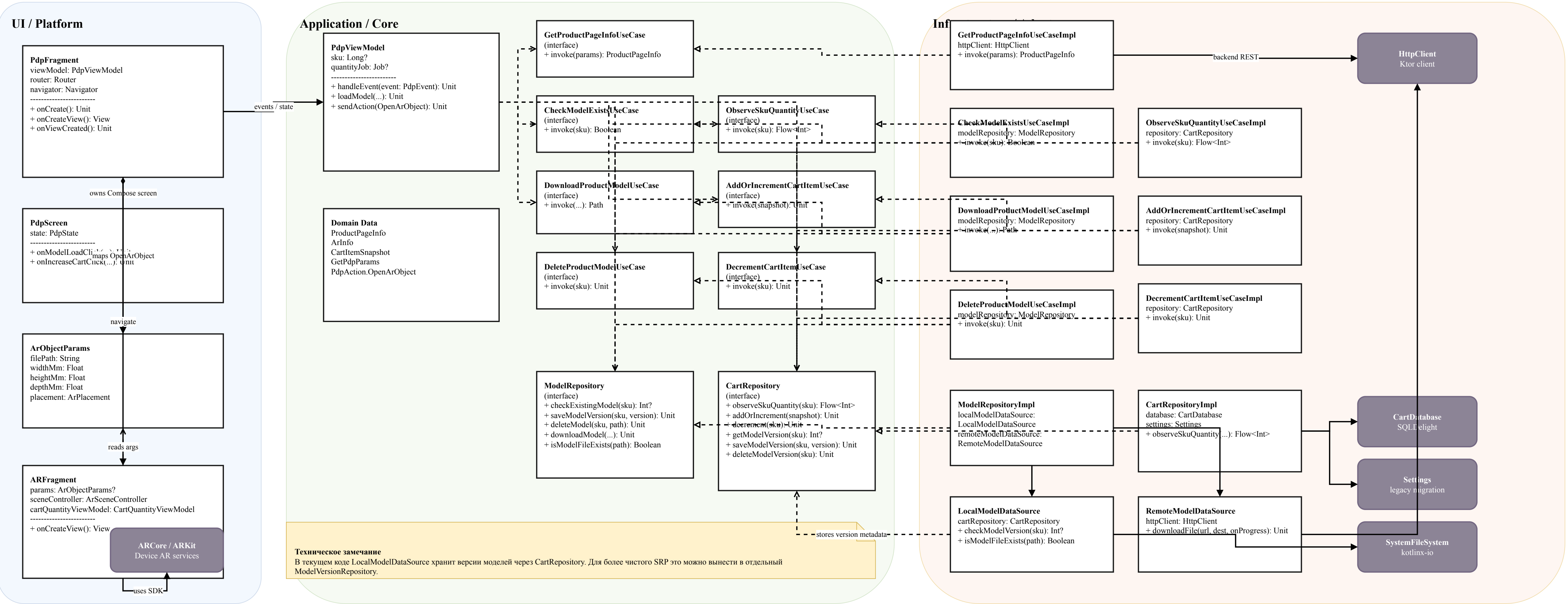


Масштабирование Ядра: UI и Infrastructure

Домашняя доработка: добавлены соседние модули, которые на соге-схеме были скрыты за интерфейсами

Архитектурная идея

UI знает только ViewModel и навигационные параметры. Concrete use case-реализации и хранилища остаются снаружи и направлены стрелками к абстракциям. Поэтому экран PDP можно тестировать без запуска SQLDelight, HTTP и ARCore/ARKit.



Обоснование, Cross-Review и Definition of Done

Домашняя доработка к блоку 3: пояснение архитектурных решений, замечания по аудиту и финальная самопроверка

Обоснование архитектуры

Для обеспечения слабой связности (loose coupling) в центре схемы оставлен PdpViewModel, который знает только контракты use case. ViewModel не создает HttpClient, репозитории, SQLDelight-таблицы или AR-фрагменты, а лишь оркестрирует сценарий: получает карточку товара, проверяет наличие 3D-модели, при необходимости инициирует загрузку, наблюдает количество в корзине и выпускает одноразовое действие OpenArObject. Благодаря этому бизнес-логика экрана не привязана к конкретной сети, БД и платформенному AR SDK.

Чтобы соблюсти принцип открытости/закрытости (OCP) и инверсию зависимостей (DIP), concrete-классы вынесены во внешний слой и направлены стрелками к интерфейсам. ModelRepositoryImpl и CartRepositoryImpl выступают плагинами, которые можно заменить другой реализацией хранения без изменения PdpViewModel. UI-модуль ограничен Fragment/Compose-слоем и объектом ArObjectParams: экран не ходит в файловую систему, БД и сеть напрямую. Такой подход также упрощает тестирование, потому что PdpViewModel можно проверять на заглушках use case без запуска ARCore, SQLDelight и HTTP-клиента.

Разделение cart-операций на отдельные use case вместо прямого доступа ViewModel к CartRepository поддерживает ISP: UI работает с узкими контрактами и не знает о лишних методах репозитория. Отдельное действие OpenArObject разрывает связь между состоянием PDP и навигацией в AR-модуль, поэтому ARFragment получает только те параметры, которые реально нужны для сцены.

Cross-Review: минимум 3 найденные проблемы

1. GetProductPageInfoUseCaseImpl напрямую зависит от HttpClient. Для текущего масштаба это допустимо как adapter, но при росте бизнес-логики стоит ввести отдельный ProductPageGateway, чтобы транспорт не просачивался в application service.
2. LocalModelDataSource хранит версии моделей через CartRepository. Это смешивает ответственность AR-кэша и корзины. Для более чистого SRP лучше выделить отдельный ModelVersionRepository / ModelCacheRepository.
3. PdpViewModel уже координирует несколько под-сценариев: загрузку PDP, кэш модели, cart quantity и навигацию в AR. Если добавить офлайн-очередь, аналитику или preload нескольких моделей, имеет смысл вынести PrepareArPreviewUseCase, чтобы ViewModel не превратился в God Object.

Definition of Done

- [x] Читаемость: core-схема и расширенная схема влезают на один экран и разделены на логические зоны.
- [x] Связи: на диаграммах нет неопределенных линий; везде показано направление зависимости или тип композиции / реализации.
- [x] SOLID (DIP): UI зависит от ViewModel и use case-контрактов, concrete-реализации направлены к абстракциям, прямого UI -> DB нет.
- [x] SOLID (ISP): экран работает через узкие контракты use case, а не через толстый общий сервис.
- [x] Именование: нет Manager/MyClassI, названия отражают роль класса и сценарий.
- [x] Типы данных: у полей и ключевых сигнатур указаны типы.
- [~] Остаточный риск: в текущем коде сохранена связка LocalModelDataSource -> CartRepository и прямой HttpClient в GetProductPageInfoUseCaseImpl; они вынесены в раздел аудита как технический долг.

Как использовать в ПЗ

1. Страница UML Core подходит как основная диаграмма классов ключевого модуля для подраздела Core Domain.
2. Страница Scaled Architecture показывает, как к ядру подключаются UI и инфраструктурные адаптеры, и помогает объяснить Clean Architecture на материале проекта.
3. Текущая страница может использоваться как текстовое приложение к разделу аудита архитектуры: она содержит обоснование, cross-review и проверку по Definition of Done.

Финальный экспорт рекомендуется хранить в SVG и PDF, чтобы диаграммы не теряли четкость при печати диплома.